

Pasteur — local stress-testing for clinical models.

Pasteur tests how a clinical model's predictions change when data goes wrong: a lab goes **missing**, a measurement is **noisy**, or a patient sits **between** two outcomes. It runs **entirely on your machine**, makes **no network calls**, and installs with standard tools.

EXECUTION 100% Local Local CPU. No server, daemon, or admin rights	NETWORK AT RUNTIME 0 Calls No telemetry, no external APIs. Runs with egress blocked	LICENCE Open source · BSD-3 pip install pypasteur · cargo install pasteur-cli
---	--	--



01 Install & run

STANDARD PYTHON & SHELL TOOLING

→ OPTION A: PYTHON (JUPYTER / SCRIPTS)

Simulate missing and noisy data. Python 3.12+, polars DataFrames.

```
pip install pypasteur polars
```

```
import polars as pl
import pypasteur

df = pl.read_parquet("cohort.parquet")

# Remove crp (and its measured flag) for 20% of patients
sim = pypasteur.BlackoutSimulator(
    "crp", rate=0.2, companions=["crp_measured"]
)
sim.fit(df)
missing = sim.transform(df)
missing.write_parquet("cohort_blackout.parquet")
```

→ OPTION B: COMMAND LINE (FULL WORKFLOW)

Simulate, then score several candidate ONNX models side by side.

```
# 1. Build blackout, jitter & flipper variants
pasteur-cli simulate --input cohort.parquet \
  --labels groups.parquet --positive-group-id 1 \
  --id-col patient_id --feature crp \
  --blackout-companion crp_measured --output ./sim

# 2. Score all candidate models on the blackout variant
mkdir -p results
pasteur-cli compare blackout --sim-root ./sim \
  --labels groups.parquet --positive-group-id 1 \
  --model models/a.onnx --model models/b.onnx \
  --predictions-out results/blackout.parquet \
  --output results/blackout.json
```

Repeat step 2 with `jitter` and `flipper`. Models from scikit-learn or PyTorch are exported to ONNX first.

02 What you get

LOCAL PARQUET & JSON

OUTPUT	MEANING
sim/<type>/*.parquet	Clean cohort plus blackout, jitter, and flipper variants. Rows keep <code>source_row_id</code>
resiliency	AUC with the feature missing + clean AUC (1.0 = no loss)
jitter_stability	How little predictions move under noise (1.0 = not at all)
flipper_stability	Where, between two opposite-label patients, the decision flips
all_agree	Per patient: do all candidate models make the same call?

03 Security & isolation

FOR YOUR REVIEW

DIMENSION	WHAT PASTEUR DOES
Location	On-device compute only
Network	No calls at runtime. Install needs PyPI/crates.io or an internal mirror; offline install is documented
Data scope	Reads and writes only the paths you pass. Logs paths and row counts, never values
Outputs	Contain patient IDs and features. Handle them like the input
Runtime	Python 3.12+ or a single CLI binary; no root

04 Example: choosing between candidate models

MODEL SELECTION

A research team has trained five candidate risk models and wants to know which stays reliable when a key lab is missing or noisy. On a secured laptop they export the models to ONNX, run `pasteur-cli simulate` once and `pasteur-cli compare` for each simulation type, with no data leaving the machine. They weigh clean accuracy against resiliency and stability, then take the specific patients the models disagree on to the clinical team for review. Pasteur provides evidence about model behaviour. It is not a medical device and does not make clinical decisions.